

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и методы программирования»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

Выполнили:

Бардышев Артём Антонович,
студент группы N3346

(подпись)

Проверил:

Ищенко Алексей Петрович,
преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

Введение.....	4
1 Страница регистрации (register.html).....	6
2 Страница авторизации (login.html)	7
3 Страница MFA (authentication.html / mfa.html)	9
4 Панель пользователя (user.html / app.html).....	10
4.1 Информация о пользователе	10
4.2 Проверка доступа (роли)	11
4.3 Обновление токенов (Refresh).....	11
4.4 Смена пароля	11
4.5 Управление MFA	12
4.6 Управление ролями (только для администратора).....	12
4.7 Управление сессиями	13
5 Внутренний API (lab6.js).....	14
5.1 Структура модуля.....	14
5.2 Безопасность (в рамках учебной работы).....	16
Заключение.....	18

ВВЕДЕНИЕ

Цель работы – Разработка системы аутентификации и авторизации с поддержкой MFA (многофакторной аутентификации), управления сессиями и ролями пользователей. Изучение современных методов обеспечения безопасности веб-приложений, включая хэширование паролей, работу с токенами, управление сессиями и реализацию многофакторной аутентификации.

Теоретическая часть

Аутентификация и авторизация

Аутентификация — процесс проверки подлинности пользователя (кто вы?).

Авторизация — процесс проверки прав доступа пользователя (что вам разрешено?).

Современные методы безопасности

Хэширование паролей

PBKDF2 (Password-Based Key Derivation Function 2) — стандарт для безопасного хэширования паролей:

- Использует криптографическую хэш-функцию (SHA-256)
- Применяет соль (salt) для защиты от rainbow tables
- Множественные итерации (120 000+) для защиты от brute-force
- Пароль никогда не хранится в открытом виде

Формула:

```
hash = PBKDF2(password, salt, SHA-256, iterations=120000)
```

Токены доступа

JWT (JSON Web Token) — стандарт для передачи информации между сторонами:

- Access Token — короткоживущий токен для доступа к ресурсам (5 минут)
- Refresh Token — долгоживущий токен для обновления access токена (24 часа)
- Структура: `header.payload.signature` (base64url)

Преимущества:

- Stateless — не требует хранения на сервере
- Масштабируемость
- Безопасность через подпись

Многофакторная аутентификация (MFA)

Принцип: Использование двух и более факторов для подтверждения личности:

1. Что-то, что вы знаете (пароль)
2. Что-то, что у вас есть (код из приложения/СМС)
3. Что-то, чем вы являетесь (биометрия)

В данной работе: 6-значный код с TTL 5 минут.

Управление сессиями

Сессия — период активности пользователя после аутентификации:

- Создание при входе
- Обновление через refresh токен
- Завершение при выходе или истечении
- Управление множественными сессиями

Описание реализованной системы

Архитектура решения

Система реализована как SPA (Single Page Application) на чистом JavaScript без серверной части. Все данные хранятся в `localStorage` браузера для учебных целей.

Работа реализована на сайте: <https://gitububkm.github.io>

Структура системы

Страницы:

└─ index.html	Главная страница
└─ register.html	Регистрация
└─ login.html	Авторизация
└─ authentication.html (mfa.html)	MFA подтверждение
└─ user.html (app.html)	Панель пользователя

JavaScript:

└─ lab6.js	Основной модуль с API
------------	-----------------------

1 СТРАНИЦА РЕГИСТРАЦИИ (REGISTER.HTML)

Назначение: Создание нового аккаунта пользователя.

Функциональность:

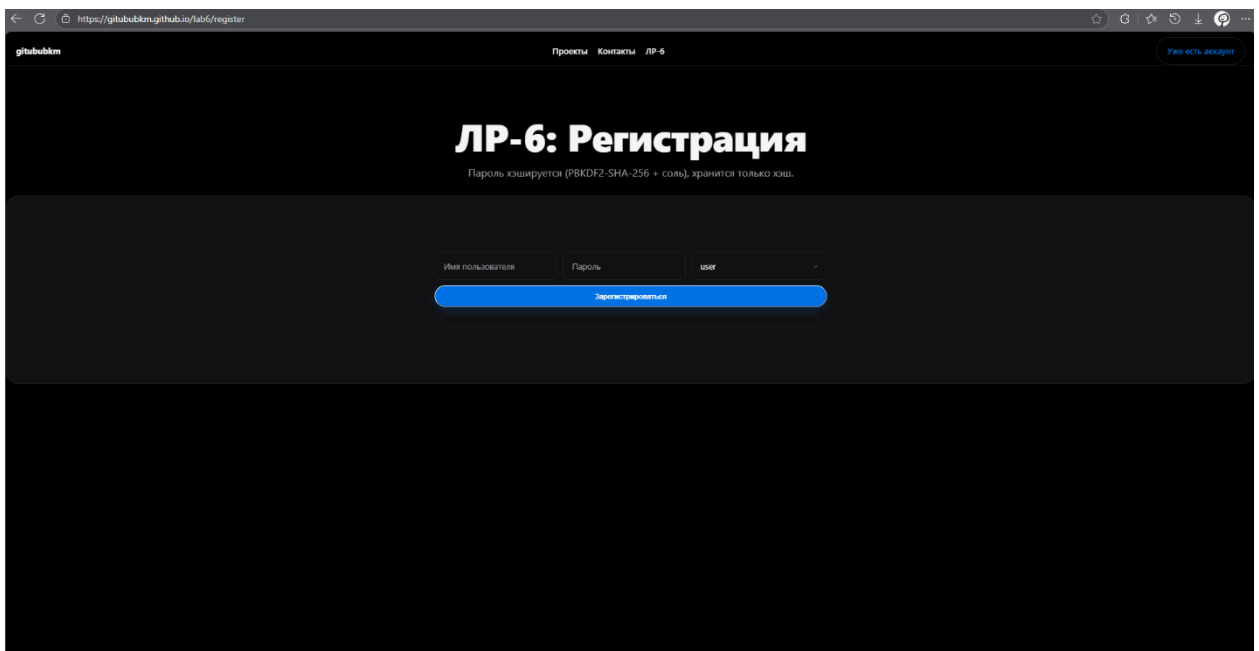
- Ввод имени пользователя (username)
- Ввод пароля (с подтверждением)
- Выбор роли (user/admin)
- Валидация данных
- Хэширование пароля с использованием PBKDF2
- Сохранение в "базу данных" (localStorage)

Алгоритм регистрации:

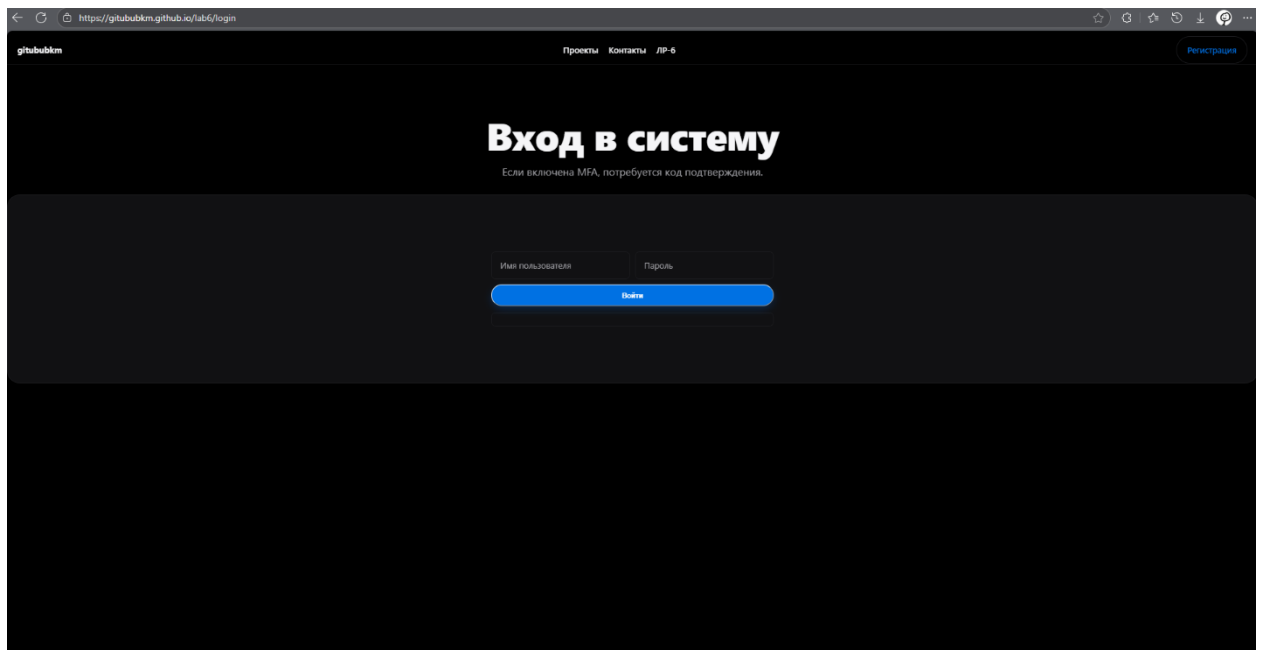
1. Пользователь вводит данные формы
2. Проверка уникальности имени пользователя
3. Генерация криптографической соли (16 байт)
4. Хэширование пароля: `PBKDF2(password, salt, SHA-256, 120000 итераций)`
5. Сохранение в localStorage:

```
{  
  "username": "user123",  
  "salt": "base64_encoded_salt",  
  "hash": "base64_encoded_hash",  
  "role": "user",  
  "mfa_enabled": false  
}
```

6. Переход на страницу авторизации



2 СТРАНИЦА АВТОРИЗАЦИИ (LOGIN.HTML)



Назначение: Вход в систему существующего пользователя.

Функциональность:

- Ввод логина и пароля
- Проверка учетных данных
- Проверка статуса MFA
- Создание сессии при успешном входе
- Редирект на MFA или панель пользователя

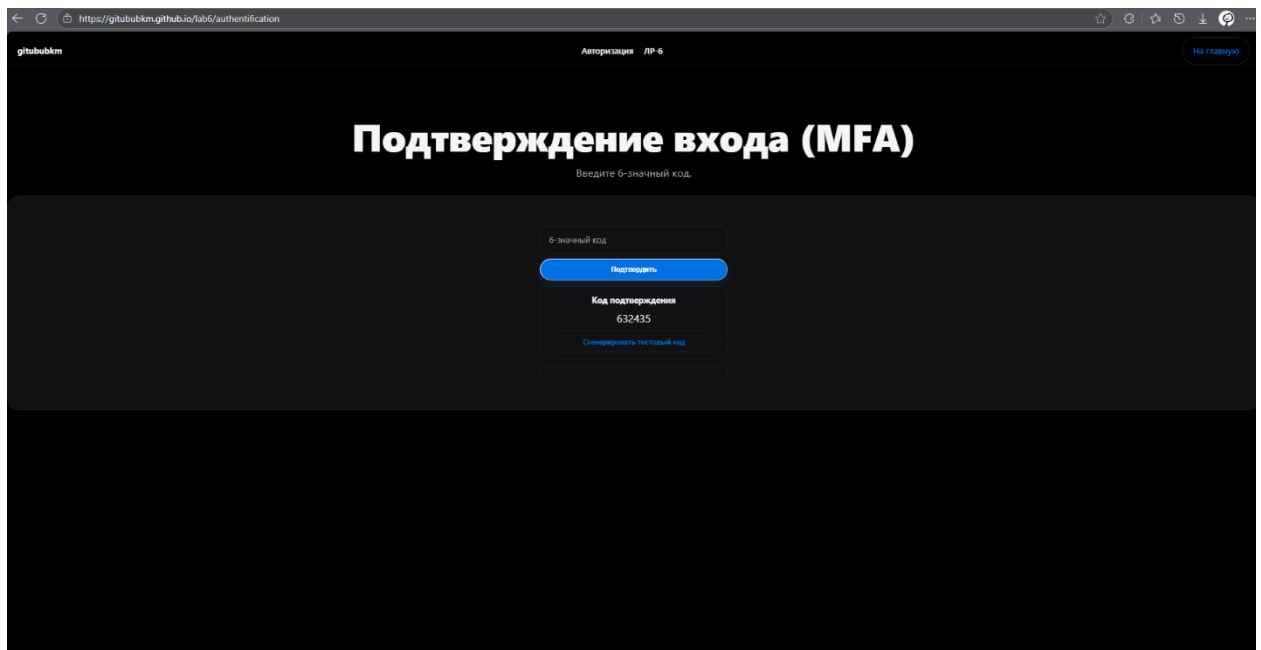
Алгоритм авторизации:

1. Пользователь вводит логин и пароль
2. Поиск пользователя в "БД" (localStorage)
3. Проверка пароля:
 - Загрузка соли и хэша пользователя
 - Хэширование введенного пароля с той же солью
 - Сравнение хэшей
4. Если пароль верен:
 - Проверка статуса MFA
 - Если MFA включена → редирект на `mfa.html`
 - Если MFA выключена → создание сессии и редирект на панель
5. Если пароль неверен → сообщение об ошибке

Создание сессии:

```
{
  "accessToken": "header.payload.signature",
  "refreshToken": "header.payload.signature",
  "accessExp": timestamp + 5 минут,
  "refreshExp": timestamp + 24 часа,
  "userId": "username",
  "role": "user",
  "sessionId": "unique_session_id"
}
```

3 СТРАНИЦА MFA (AUTHENTICATION.HTML / MFA.HTML)



Назначение: Подтверждение входа через многофакторную аутентификацию.

Функциональность:

- Отображение 6-значного кода (в демо-версии)
- Ввод кода пользователем
- Проверка кода и TTL
- Создание полноценной сессии при успехе

Алгоритм MFA:

1. Генерация 6-значного кода (случайное число 100000-999999)
2. Сохранение кода с временной меткой (TTL 5 минут)
3. Отображение кода на странице (в демо-версии)
4. Пользователь вводит код
5. Проверка:
 - Совпадение кода
 - Не истек ли срок действия (5 минут)
6. При успехе → создание сессии и редирект на панель
7. При неудаче → сообщение об ошибке

В реальной системе:

- Код отправляется через SMS, email или приложение-аутентификатор
- В демо-версии код показывается на странице для тестирования

4 ПАНЕЛЬ ПОЛЬЗОВАТЕЛЯ (USER.HTML / APP.HTML)

Назначение: Основная рабочая область системы после входа.

Функциональность:

4.1 Информация о пользователе

- Отображение имени пользователя
- Текущая роль (user/admin)
- Статус MFA (включена/выключена)
- Информация о текущей сессии

Панель пользователя
Роли, refresh, смена пароля, MFA и управление сессиями.

Пользователь: root1
Роль: admin
Access: выдан
Refresh: выдан

Доступ
Проверка авторизации по роли.

USER
ADMIN
Обновить все

Смена пароля
Все сессии будут сброшены.

Старый пар Новый пар
Сменить

MFA

Включить
Выключить

Роли (admin)

Имя поль user
Назначить

Сессии

Показать
Завершить текущую
Завершить все

4.2 Проверка доступа (роли)

Кнопки доступа:

- USER — доступен всем пользователям
- ADMIN — доступен только администраторам

Принцип работы:

- При нажатии на кнопку проверяется роль пользователя
- Если доступ разрешен → отображается контент
- Если доступ запрещен → сообщение об ошибке

4.3 Обновление токенов (Refresh)

Функциональность:

- Отображение времени жизни access токена
- Кнопка "Обновить access токен"
- Обновление истекшего access токена через refresh токен
- Проверка валидности refresh токена

Алгоритм обновления:

1. Проверка валидности refresh токена
2. Если refresh токен валиден:
 - Генерация нового access токена
 - Обновление времени жизни
 - Сохранение в сессию
3. Если refresh токен истек → требование повторного входа

4.4 Смена пароля

Функциональность:

- Ввод старого пароля
- Ввод нового пароля (с подтверждением)
- Проверка старого пароля
- Генерация новой соли и хэша
- Инвалидация всех сессий пользователя

Алгоритм смены пароля:

1. Пользователь вводит старый пароль
2. Проверка старого пароля (сравнение хэшей)

3. Если старый пароль верен:

- Генерация новой соли
- Хэширование нового пароля
- Обновление данных пользователя
- Инвалидация всех активных сессий
- Требование повторного входа

4. Если старый пароль неверен → ошибка

4.5 Управление MFA

Функциональность:

- Включение/выключение MFA
- Отображение текущего статуса
- Предупреждения о последствиях

Алгоритм:

1. Переключение статуса MFA (вкл/выкл)
2. Сохранение настройки в профиле пользователя
3. При следующем входе будет использоваться новый статус

4.6 Управление ролями (только для администратора)

Функциональность:

- Список всех пользователей
- Назначение ролей (user/admin)
- Изменение ролей других пользователей

Алгоритм:

1. Загрузка списка всех пользователей
2. Отображение текущей роли каждого пользователя
3. Выбор пользователя и новой роли
4. Сохранение изменений
5. Изменения вступают в силу при следующем входе пользователя

4.7 Управление сессиями

Функциональность:

- Просмотр списка активных сессий
- Информация о каждой сессии:
 - ID сессии
 - Время создания
 - Время истечения access токена
 - Время истечения refresh токена
 - IP-адрес (если доступен)
- Завершение текущей сессии
- Завершение всех сессий

5 ВНУТРЕННИЙ API (LAB6.JS)

5.1 Структура модуля

Модуль `LAB6` предоставляет все необходимые методы для работы системы:

```
const LAB6 = {  
  // Регистрация и вход  
  register(username, password, role),  
  login(username, password),  
  // MFA  
  peekMFACode(),  
  verifyMFA(code),  
  // Сессии  
  current(),  
  refresh(),  
  logout(),  
  // Авторизация  
  call(scope),  
  // Управление  
  changePassword(old, neu),  
  setMFA(onOrObj),  
  setRole(username, role),  
  listSessions(),  
  killCurrent(),  
  killAllMine()  
};
```

Описание методов

Регистрация и аутентификация

`LAB6.register(username, password, role)`

- Регистрация нового пользователя
- Генерация соли и хэширование пароля (PBKDF2)
- Сохранение в localStorage
- Возвращает: `true` при успехе, `false` при ошибке

`LAB6.login(username, password)`

- Проверка учетных данных
- Создание сессии (если MFA выключена)
- Возвращает: `mfa` (если MFA включена), `ok` (успех), `error` (ошибка)

MFA

`LAB6.peekMFACode()`

- Получение сгенерированного MFA кода (для демо)
- Возвращает: 6-значный код

`LAB6.verifyMFA(code)`

- Проверка MFA кода
- Создание полноценной сессии при успехе

- Возвращает: `true` (успех), `false` (ошибка)

Управление сессиями

`LAB6.current()`

- Получение информации о текущей сессии
- Возвращает: объект с данными сессии или `null`

`LAB6.refresh()`

- Обновление access токена через refresh токен
- Возвращает: `true` (успех), `false` (ошибка)

`LAB6.logout()`

- Завершение текущей сессии
- Очистка данных из localStorage

Авторизация

`LAB6.call(scope)`

- Проверка доступа к ресурсу
- `scope = 'user'` — доступен всем
- `scope = 'admin'` — только администраторам
- Возвращает: `true` (доступ разрешен), `false` (доступ запрещен)

Управление

`LAB6.changePassword(old, neu)`

- Смена пароля пользователя
- Проверка старого пароля
- Генерация новой соли и хэша
- Инвалидация всех сессий
- Возвращает: `true` (успех), `false` (ошибка)

`LAB6.setMFA(onOrObj)`

- Включение/выключение MFA
- `onOrObj = true/false`
- Сохранение в профиле пользователя

`LAB6.setRole(username, role)`

- Назначение роли пользователю (только для администратора)
- `role = 'user' | 'admin'`
- Возвращает: `true` (успех), `false` (ошибка)

`LAB6.listSessions()`

- Получение списка всех активных сессий пользователя
- Возвращает: массив объектов сессий
- `LAB6.killCurrent()`
- Завершение текущей сессии
- Возвращает: `true`
- `LAB6.killAllMine()`
- Завершение всех сессий текущего пользователя
- Возвращает: количество завершенных сессий

5.2 Безопасность (в рамках учебной работы)

Хранение паролей

Алгоритм:

```
// Генерация соли (16 байт)
const salt = crypto.getRandomValues(new Uint8Array(16));
// Хэширование пароля
const keyMaterial = await crypto.subtle.importKey(
  'raw',
  new TextEncoder().encode(password),
  'PBKDF2',
  false,
  ['deriveBits']
);
const hash = await crypto.subtle.deriveBits(
  {
    name: 'PBKDF2',
    salt: salt,
    iterations: 120000,
    hash: 'SHA-256'
  },
  keyMaterial,
  256
);
```

Хранение:

- В localStorage сохраняются только `salt` (base64) и `hash` (base64)
- Пароль никогда не сохраняется в открытом виде
- Каждый пользователь имеет уникальную соль

Токены

Структура JWT-подобного токена:

header.payload.signature

Header:

```
{
  "alg": "HS256",
  "typ": "JWT"
},
...
```

Payload:

```json

```
{
 "sub": "username",
 "role": "user",
 "sid": "session_id",
 "exp": 1735068000
}
```

Особенности:

- Access token: время жизни 5 минут
- Refresh token: время жизни 24 часа
- В учебной версии подпись не проверяется (для упрощения)

Сессии

Структура сессии:

```
{
 "sessionId": "unique_id",
 "userId": "username",
 "role": "user",
 "accessToken": "token_string",
 "refreshToken": "token_string",
 "accessExp": 1735068000,
 "refreshExp": 1735154400,
 "createdAt": 1735067700
}
```

Хранение:

- В localStorage: `lab6\_current\_session` — текущая сессия
- В "БД": `lab6\_sessions[sessionId]` — все активные сессии

MFA

Генерация кода:

```
const code = Math.floor(100000 + Math.random() * 900000); // 100000-999999
```

Хранение:

- Код сохраняется с временной меткой
- TTL: 5 минут
- После использования или истечения — удаление

Смена пароля

Процесс:

1. Проверка старого пароля
2. Генерация новой соли
3. Хэширование нового пароля
4. Обновление данных пользователя
5. Инвалидация всех сессий пользователя
6. Требование повторного входа



## ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены и реализованы:

Теоретические знания

1. Принципы аутентификации и авторизации:
  - Различие между аутентификацией и авторизацией
  - Методы проверки подлинности
  - Системы ролей и прав доступа
2. Безопасное хранение паролей:
  - Алгоритм PBKDF2
  - Использование криптографической соли
  - Защита от rainbow tables и brute-force
3. Работа с токенами:
  - JWT-подобные токены
  - Access и Refresh токены
  - Механизм обновления токенов
4. Многофакторная аутентификация:
  - Принципы MFA
  - Генерация и проверка кодов
  - TTL и безопасность кодов
5. Управление сессиями:
  - Создание и хранение сессий
  - Множественные сессии
  - Инвалидация сессий

Практические навыки

1. Разработка веб-приложений:
  - Создание SPA на чистом JavaScript
  - Работа с localStorage
  - Управление состоянием приложения
2. Криптография:
  - Использование Web Crypto API
  - Реализация PBKDF2
  - Работа с токенами

### 3. Безопасность:

- Реализация безопасного хранения паролей
- Защита от несанкционированного доступа
- Управление сессиями

Ограничения учебной версии